📒 Mi chuletario — Examen Python

> Apuntes para el examen, explicados para entenderlos aunque me haya olvidado. **Cómo buscar rápido:** `Ctrl+F` + una palabra, o mira el índice / la tabla de abajo.

Índice

1. Leer código: ¿qué se muestra?
2. Comprehensions
3. Programación funcional (lambda, map, filter, reduce)
4. Recursión
5. Clases y objetos
6. Sobrecarga de operadores
7. Herencia, polimorfismo y MRO
8. Refactorizar if/elif a clases
9. Integrador (mezcla de todo)
10. Programación modular
11. ⚡ Trucos: f-strings y redondeo
12. ❗ Errores típicos y qué significan

🔎 Busca rápido por palabra

Si veo en el enunciado…	Voy a
"¿qué muestra / imprime?"	§1
"una sola list comprehension", `[... for ...]`, set/dict comp	§2
`lambda`, `map`, `filter`, `reduce`, `sorted`, función que recibe/devuelve función	§3
"función recursiva", "sin bucles", caso base	§4
"crea una clase", `__init__`, método, `@classmethod`, `@staticmethod`, composición	§5
`__add__`, `__eq__`, `__len__`, `__str__`, `+`, `==`, `in`, `+=`	§6
"hereda de", `super()`, `__mro__`, polimorfismo	§7
"refactoriza", quitar `if/elif` por tipo	§8
ejercicio largo que mezcla todo	§9
módulo, paquete, `import`, `__init__.py`, `@property`, `if __name__`	§10
2 decimales, alinear columnas, `round`, `//`, `%`	⚡ Trucos
nombre de un error (`TypeError`, `ValueError`…)	❗ Errores

---

1. Leer código: ¿qué se muestra?

Qué me piden: leer un código y decir qué imprime, sin ejecutarlo. No hay que programar.

Atributo de clase vs. de instancia

```python
class Tienda:
    total = 0                  # ATRIBUTO DE CLASE: lo comparten todas las tiendas
    def __init__(self, nombre):
        self.nombre = nombre   # ATRIBUTO DE INSTANCIA: propio de cada objeto
        Tienda.total += 1      # subo el contador de la CLASE
```

- `Tienda.total` → el valor compartido.
- `t1.total = 10` **NO** cambia el de la clase: crea un atributo propio de `t1` que "tapa" al de la clase. Los demás objetos siguen viendo el de la clase.
- **Trampa:** atributo de clase **mutable** (`lista = []` en la clase) → lo comparten TODOS los objetos; un `.append` desde uno lo ve el resto.

Listas: copia vs. mismo objeto

```
b = a        # b y a son EL MISMO objeto. Cambiar b cambia a.
c = a[:]     # c es una COPIA nueva (también a.copy() o list(a)).
```

- `a is b` → ¿son el mismo objeto? · `a == b` → ¿tienen el mismo contenido?

Argumento por defecto mutable (trampa clásica)

```
def añadir(x, mochila=[]):     # ⚠️ la lista por defecto se CREA UNA vez y se REUTILIZA
    mochila.append(x)
    return mochila
añadir("a")   # ['a']
añadir("b")   # ['a', 'b']  ← ¡arrastra la anterior!
```

Solución correcta: `def añadir(x, mochila=None): if mochila is None: mochila = []`.

Guiones bajos en atributos

- `self.titular` → público, se usa desde fuera.
- `self._saldo` → un guion: "no lo toques desde fuera" (solo convención, sí se puede leer).
- `self.__pin` → dos guiones: Python lo renombra a `_Clase__pin` (name mangling). Desde fuera `obj.__pin` da **`AttributeError`**; hay que escribir `obj._Clase__pin`.

Closures (función que recuerda variables)

```
def crear(prefijo):
    def interna(x):
        return prefijo + "-" + str(x)   # RECUERDA prefijo
    return interna
f = crear("FAC")      # f(15) → "FAC-15"
```

- En `factura(albaran(1))` primero se calcula lo de **dentro**: `albaran(1)`, y su resultado entra en `factura`.
- Con `nonlocal total` dentro de la función interna, el closure **guarda un estado** entre llamadas (un contador). Dos closures distintos (`c1`, `c2`) tienen cada uno el suyo.

Ojo con: recursión sin caso base → `RecursionError`. `print(map(...))` sin `list()` → sale `<map object>`.

---

2. Comprehensions

La idea: un bucle que crea una lista en una línea. `[x*2 for x in lista]` = recorrer la lista y guardar `x*2`. Tres piezas: `[QUÉ GUARDO for x in DÓNDE if FILTRO]`.

Moldes:

```
[EXPR for x in lista]                 # transformar cada elemento
[EXPR for x in lista if COND]         # FILTRAR: el if va AL FINAL, sin else
[A if COND else B for x in lista]     # CLASIFICAR: el if/else va DELANTE, con else
{EXPR for x in lista}                 # SET (sin repetidos)
{CLAVE: VALOR for x in lista}         # DICCIONARIO
[y for sub in lista for y in sub]     # DOS for: aplanar listas de listas
```

La diferencia clave (cae siempre) — la posición del `if`: - `if` **al final** = filtro, decide qué entra. La lista puede quedar más corta. **Sin `else`.** - `if ... else` **delante** = decide qué valor pongo para cada elemento. **Con `else`.** Misma longitud. - Para leer una comp. ajena: fíjate primero en el `for` (qué recorro), luego el principio (qué guardo) y el final (si filtro).

Tres opciones encadenadas:

```
"COMPLETA" if n == 0 else "ÚLTIMAS" if n < 10 else "DISPONIBLE"
```

Útiles: `lista.count("X")` cuenta apariciones · `texto.strip()` quita espacios · `texto.capitalize()` primera en mayúscula · recorrer diccionario: `for clave, valor in d.items():`

Ojo con: - Hay que **guardar** el resultado: `resultado = [...]`. Si no lo asignas, se pierde. - Recorre la lista de **datos**, no una vacía; usa un nombre de variable distinto al de la lista (`for p in precios`, no `for precios in precios`). - `round(x, 2)` redondea a 2 decimales (el `2` es el nº de decimales, **no** divide). "precio con IVA 21%" = `p * 1.21`.

```
datos = [8.5, 12.0, 45.9]
resultado = [round(p * 1.21, 2) for p in datos if p > 10]   # correcto
```

---

3. Programación funcional (lambda, map, filter, reduce)

`lambda` = función corta en una línea:

```
doble = lambda x: x * 2            # doble(3) → 6
es_grande = lambda x: x >= 10      # devuelve True/False
```

`map` (transformar), `filter` (seleccionar): aplican una función a cada elemento. Devuelven un iterador → **envolver en `list()`** (si no, sale `<map object>`).

```
list(map(lambda c: c["km"], carreras))                      # saca un campo de cada dict
list(filter(lambda c: c["tarjeta"] and c["km"] >= 10, datos))# deja los que cumplen
```

`map(f, lista)` es lo mismo que `[f(x) for x in lista]`; usa el que te pida el enunciado.

`sorted` (ordenar) y `max`/`min` por un campo:

```
sorted(carreras, key=lambda c: c["km"], reverse=True)   # reverse=True: de mayor a menor
max(carreras, key=lambda c: c["km"])                    # el que más km tiene
```

`reduce` (acumular en un solo valor): hay que importarlo.

```
from functools import reduce
# SUMAR un campo → valor inicial 0:
total = reduce(lambda acum, c: acum + c["importe"], carreras, 0)
# EL MAYOR → sin valor inicial, comparando parejas:
mayor = reduce(lambda a, b: a if a["km"] >= b["km"] else b, carreras)
# filter + reduce combinados:
km = reduce(lambda acum, c: acum + c["km"], filter(lambda c: c["tarjeta"], carreras), 0)
# acumular en un DICCIONARIO (avanzado) → valor inicial {}:
def acum(d, v):
    d[v["cliente"]] = d.get(v["cliente"], 0) + v["importe"]
    return d
por_cliente = reduce(acum, ventas, {})
```

Función que RECIBE una función:

```
def transformar(lista, operacion):
    return [operacion(x["importe"]) for x in lista]
transformar(datos, a_dolares)          # se pasa SIN paréntesis
transformar(datos, lambda x: x * 1.1)
```

Función que DEVUELVE una función:

```
def crear_tarifa(precio_km):
    def calcular(km):
        return round(km * precio_km, 2)
    return calcular                    # SIN paréntesis
nocturna = crear_tarifa(1.35)          # nocturna(10) → 13.5
```

Ojo con: dentro de un f-string con comillas dobles, usa comillas simples para la clave: `f"Carrera {c['id']}"`.

---

4. Recursión

Receta (siempre): 1. **Caso base:** cuándo paro y qué devuelvo (lista vacía, `n` pequeño…). 2. **Caso recursivo:** un trozo + la función sobre algo **más pequeño**.

```python
def suma_lista(l):                  # recorrer lista
    if not l: return 0              # base: vacía
    return l[0] + suma_lista(l[1:])

def suma_digitos(n):                # número cifra a cifra
    if n < 10: return n
    return n % 10 + suma_digitos(n // 10)   # n%10 última cifra, n//10 la quita

def es_palindromo(t):               # texto por los extremos
    if len(t) <= 1: return True
    if t[0] != t[-1]: return False
    return es_palindromo(t[1:-1])

def ahorro(capital, anios):         # repetir n veces (interés 2%)
    if anios == 0: return capital
    return ahorro(capital * 1.02, anios - 1)

def contar_unidades(pedidos):       # campo de lista de diccionarios
    if not pedidos: return 0
    return pedidos[0]["unidades"] + contar_unidades(pedidos[1:])

def peso_total(caja):               # listas anidadas: AQUÍ sí vale un for
    total = 0
    for e in caja:
        if isinstance(e, list):     # es otra caja → recursión
            total += peso_total(e)
        else:                       # es un artículo (para CONTAR: total += 1)
            total += e
    return total
```

Pregunta típica: con muchas llamadas (ej. `anios = 5000`) da **`RecursionError`** (límite ≈ 1000). La versión iterativa (con bucle) no tiene ese problema.

---

5. Clases y objetos

```python
class Cuenta:
    total_cuentas = 0                       # atributo de CLASE (contador)
    COMISION = 1.5                          # constante de clase

    def __init__(self, titular, saldo=0.0):   # saldo por defecto 0.0
        self.titular = titular
        self._saldo = saldo
        Cuenta.total_cuentas += 1           # ¡con Cuenta., NO self.!

    def retirar(self, importe):             # devuelve True/False
        if importe + Cuenta.COMISION <= self._saldo:
            self._saldo -= importe + Cuenta.COMISION
            return True
        return False

    @classmethod
    def cuentas_creadas(cls):               # recibe la CLASE (cls)
        return cls.total_cuentas

    @classmethod
    def desde_pares(cls, pares):            # CONSTRUCTOR alternativo: devuelve un objeto ya montado
        obj = cls()
        # ... rellenar obj ...
        return obj

    @staticmethod
    def desde_texto(t):                     # NO recibe self ni cls
        return Cuenta("X", float(t.replace(",", ".")))

    def __str__(self):                      # print(obj) → texto para personas
        return f"{self.titular}: {self._saldo:.2f} €"
    def __repr__(self):                     # en listas/consola → para programadores
        return f"Cuenta({self.titular!r}, {self._saldo})"   # !r pone comillas
```

- `__str__`/`__repr__` **devuelven** texto (`return`), no imprimen.
- `print(obj)` usa `__str__`; si no existe, usa `__repr__`.

Composición (una clase CONTIENE objetos de otra):

```python
class Carrito:
    def __init__(self):
        self.lineas = []                    # lista de tuplas (articulo, cantidad)
    def agregar(self, articulo, cantidad):
        self.lineas.append((articulo, cantidad))
    def total(self):
        return sum(a.precio * c for a, c in self.lineas)
```

Lista de objetos en una línea:

```python
[a.nombre for a in catalogo if a.stock == 0]
{a.nombre: a.valor_inventario() for a in catalogo}
sorted(catalogo, key=lambda a: a.precio)
```

---

6. Sobrecarga de operadores

Hacer que `+`, `==`, `len()`, `in`... funcionen sobre MIS objetos. Cada operador tiene su método mágico:

Quiero	Método	Devuelve
`a + b`	`__add__(self, otro)`	**objeto nuevo**
`a * n`	`__mul__(self, n)`	**objeto nuevo**
`a == b` / `a < b`	`__eq__` / `__lt__`	bool
`str(a)` / `print(a)`	`__str__`	str
`float(a)` / `bool(a)` / `len(a)`	`__float__` / `__bool__` / `__len__`	número/bool/int
`x in a`	`__contains__(self, x)`	bool
`a += x`	`__iadd__(self, x)`	**`return self`** ⚠️

```python
class Importe:
    def __init__(self, cantidad):
        self.cantidad = cantidad
    def __add__(self, otro):
        return Importe(self.cantidad + otro.cantidad)   # NUEVO objeto
    def __eq__(self, otro):
        return round(self.cantidad, 2) == round(otro.cantidad, 2)
    def __lt__(self, otro):
        return self.cantidad < otro.cantidad
    def __str__(self):
        return f"{self.cantidad:.2f} €"
    def __float__(self):
        return float(self.cantidad)
```

Ojo con: si `__iadd__` no hace `return self`, tras `a += x` la variable pasa a valer `None`.

Texto `"12,50 €"` → número: `float(texto.replace("€", "").strip().replace(",", "."))`.

---

7. Herencia, polimorfismo y MRO

```python
class Vehiculo:
    def __init__(self, matricula, km):
        self.matricula = matricula; self.km = km
    def peaje(self):
        return self.km * 0.10

class Camion(Vehiculo):
    def __init__(self, matricula, km, ejes):
        super().__init__(matricula, km)      # el padre guarda lo suyo
        self.ejes = ejes                     # yo añado lo nuevo
    def peaje(self):                         # SOBRESCRIBIR ampliando
        return super().peaje() + 2 * self.ejes
```

- **Polimorfismo:** en un bucle `for v in vehiculos: v.peaje()` cada objeto usa SU versión, sin ningún `if` que pregunte el tipo. (Respuesta a "¿dónde está el polimorfismo?": en la llamada del bucle.)
- Nombre de la clase del objeto: `type(self).__name__`.

Herencia múltiple y MRO (lo más difícil — método en 3 pasos)

1. **Escribe el MRO:** la clase, sus padres **de izquierda a derecha**, y el abuelo común **al final**, una sola vez. Compruébalo: `[c.__name__ for c in Clase.__mro__]`. `class D(B, C)` → MRO `D → B → C → A → object`.
2. `super()` **NO es "mi padre"**, es **"la SIGUIENTE clase del MRO"**. En un `D`, el `super()` de `B` llama a `C`.
3. Si cada método hace **primero** `super()` y **después** añade lo suyo (`+ ["x"]`, un `print`…), lo añadido sale en **orden INVERSO al MRO**: primero `A`, luego `C`, `B`, `D`.
4. **Por qué el abuelo aparece una sola vez:** está una vez en el MRO y la cadena de `super()` recorre el MRO.
5. **Si una clase NO llama a `super()`:** corta la cadena y las de detrás en el MRO no se ejecutan (se pierde su parte).
6. **Si cambio el orden de los padres:** cambia el MRO y con él el orden de los resultados.

Ejemplo: `Administrador(Editor, Moderador)` → MRO `Administrador → Editor → Moderador → Usuario`. Constructores que imprimen después de `super()` → salen `Usuario, Moderador, Editor, Administrador`.

---

8. Refactorizar if/elif a clases

De esto (`if r[0]=="vuelo" ... elif "hotel" ...`, repetido en varios sitios) **a esto:**

```
class Reserva:                                  # lo común
    def __init__(self, cliente, destino, cantidad, precio_unitario):
        self.cliente=cliente; self.destino=destino
        self.cantidad=cantidad; self.precio_unitario=precio_unitario
    def precio_base(self):
        return self.cantidad * self.precio_unitario
    def precio_total(self):                     # por defecto
        return self.precio_base()
    def unidad(self):
        return "unidades"
    def __str__(self):
        tipo = type(self).__name__.replace("Reserva", "").upper()  # "VUELO"
        return f"{tipo} | {self.cliente} | {self.destino} | {self.cantidad} {self.unidad()} | {round(self.

class ReservaHotel(Reserva):                    # cada tipo: SOLO lo que cambia
    def unidad(self): return "noches"
    def precio_total(self):
        precio = self.precio_base()
        if self.cantidad >= 3: precio *= 0.9    # regla del hotel, NO pregunta el tipo → vale
        return precio + self.cantidad * 1.5

class ReservaTren(Reserva):                     # no cambia el precio → hereda el del padre
    def unidad(self): return "billetes"
```

- **Reglas:** nada de `if` que pregunte el tipo, ni `isinstance` para calcular; cada cálculo en **un solo sitio**.
- **Añadir un tipo nuevo (crucero):** solo creo `class ReservaCrucero(Reserva)` nueva; no toco `Reserva` ni `Agencia`. Eso es la ventaja: abierto a ampliar, cerrado a modificar.
- `Agencia` **contiene** la lista y suma con `sum(r.precio_total() for r in self.reservas)` ← polimorfismo.

---

9. Integrador (mezcla de todo)

Mismo patrón: jerarquía de clases con `precio_final()` sobrescrito, y encima comprehensions + funcional + recursión sobre una lista de diccionarios.

```
class Plato:
    def __init__(self, nombre, precio): self.nombre=nombre; self.precio=precio
    def precio_final(self): return self.precio
    def __str__(self): return f"{self.nombre} - {self.precio_final():.2f} €"
class PlatoVegano(Plato):
    def precio_final(self): return super().precio_final() * 0.9
```

- Importe de un pedido: `pedido["plato"].precio_final() * pedido["unidades"]`.
- Filtrar por tipo de clase: `filter(lambda p: isinstance(p["plato"], PlatoVegano), pedidos)`.
- **Pregunta final típica:** añadir `PlatoInfantil` solo exige crear la subclase; todo lo que usa `precio_final()` sigue igual. Solo lo que usa `isinstance(tipo concreto)` depende de las clases.

---

10. Programación modular

- **Módulo** = un archivo `.py`. **Paquete** = carpeta con `__init__.py`.
- `if __name__ == "__main__":` → ese bloque se ejecuta SOLO si lanzo el archivo directamente (`python modulo.py`), **no** al importarlo. Al importar, `__name__` vale el nombre del módulo.
- Un módulo se ejecuta **una sola vez** aunque lo importe varias (se guarda en `sys.modules`).

Formas de importar:

```
import modulo                 # modulo.funcion()
import modulo as m            # m.funcion()
from modulo import f, CONST   # f()  (directo)
from modulo import *          # ⚠️ evitar: puede pisar mis nombres
```

Paquete (imports relativos con el punto):

```
# dentro del paquete etl, en transformacion.py:
from .config import TASA        # "." = del mismo paquete
from .modelos import Reserva
# desde main.py (fuera del paquete): import ABSOLUTO
from etl import ejecutar_pipeline, exportar_csv
```

- `__init__.py` marca la carpeta como paquete y "sube" nombres: por él puedo hacer `from etl import X` en vez de `from etl.modelos import X`. Suele llevar `__version__` y `__all__`.
- **Sin imports circulares:** el sentido va `config → modelos → transformacion/carga → orquestador → __init__ → main`.

`@property` (atributo que se calcula al pedirlo, sin paréntesis):

```
class Reserva:
    @property
    def importe_total(self):
        base = self.noches * self.precio_noche
        if self.es_socio: base *= 0.90
        return round(base + 2.50 * self.noches, 2)
# se usa:  r.importe_total   (sin paréntesis, como si fuera un atributo)
```

Función pura: no imprime ni cambia nada de fuera; mismo input → mismo output. Las de `transformacion.py` deben serlo.

⚡ Trucos: f-strings y redondeo

- `f"{x:.2f}"` → 2 decimales · `f"{x:.1f}"` → 1 decimal.
- `f"{x!r}"` → con comillas (para `'Laura'`).
- Alinear en columnas: `f"{h:<17}"` izquierda ancho 17 · `f"{n:>3}"` derecha ancho 3 · `f"{t:>9.2f}"` derecha, 2 decimales.
- `round(2.5)` → `2` y `round(3.5)` → `4` (redondea al par). Con decimales de coma flotante, `round(x, 2)` arregla comparaciones.
- `7 // 2` → 3 (entera) · `7 % 2` → 1 (resto) · `7 / 2` → 3.5.

❗ Errores típicos y qué significan

Error	Causa habitual
`AttributeError`	atributo/método que no existe (ej. `obj.__pin` con name mangling)
`RecursionError`	recursión sin caso base o demasiadas llamadas
`TypeError`	mezclar tipos, p.ej. `"a" + 3` → usa `str(3)`; o faltan/sobran argumentos
`ValueError`	`int("3.5")` o `int("80.00")` (letra O en vez de 0)
`KeyError` / `IndexError`	clave que no está en el dict / índice fuera de la lista

---

Mis dudas del estudio

(aún no hay dudas nuevas sin integrar — las que han ido saliendo ya están en sus secciones de arriba)